

rPBFT: Reliable Practical Byzantine Fault Tolerance Mechanism for Faulty Distributed Networks

Eunyoung Lee , Namryeong Kim , Chaerim Han, Nayeon Shin, and Ilgu Lee , *Member, IEEE*

Abstract—The adoption of blockchain technology across financial, industrial, and IoT ecosystems has increased the demand for reliable and scalable consensus mechanisms. Traditional Practical Byzantine Fault Tolerance (PBFT) offers strong consistency and finality but scales poorly due to high message complexity and communication overhead. To address this, we propose Reliable Practical Byzantine Fault Tolerance (rPBFT), a consensus mechanism that improves performance and fault tolerance in large-scale distributed systems. rPBFT evaluates node reliability through message exchanges during consensus and dynamically forms a committee based on reliability scores. This reduces communication load, improves scalability, and prevents performance degradation caused by faulty nodes while maintaining decentralization without representative election. Experiments evaluated rPBFT against PBFT and HotStuff. In fault-free environments, PBFT achieved better performance due to the absence of reliability computation overhead. Under faulty conditions, rPBFT outperformed PBFT and HotStuff, achieving 30%–40% lower latency, 50%–80% higher throughput, and a 33% improvement in consensus success rate. In fault-tolerance tests, rPBFT maintained stable throughput as the Byzantine ratio and network fault ratio increased, demonstrating resilience via a reliability-based recovery model. These results confirm that rPBFT enhances scalability and robustness while preserving the core properties of Byzantine fault-tolerant consensus, making it well-suited for large-scale blockchain and IoT environments.

Index Terms—Blockchain, consensus algorithm, distributed consensus mechanism, practical Byzantine fault tolerant (PBFT).

I. INTRODUCTION

BLOCKCHAIN systems rely on distributed consensus to ensure data consistency and reliability in environments where trust among nodes cannot be assumed. Unlike traditional centralized systems, where a trusted authority guarantees data integrity, blockchain networks require that all honest nodes agree on the same system state even in the presence of diverse node failures. Without an efficient consensus protocol, the core values of blockchain, namely immutability, reliability, and decentralization, cannot be achieved [1], [2].

To address these challenges, Byzantine fault tolerance (BFT) mechanisms, particularly practical Byzantine fault tolerance (PBFT), have been widely adopted owing to their high throughput, low latency, and immediate finality. PBFT ensures reliable consensus in a network with $3f+1$ nodes, where f denotes the maximum number of faulty nodes [3], [4]. However, as blockchain networks scale, PBFT faces key limitations: scalability bottlenecks caused by quadratic message complexity $O(n^2)$, vulnerability to performance collapse when more than one-third of nodes fail, dependence on a single leader as a potential point of failure, high message overhead due to its multi-phase process, and a static fault model that cannot adapt to dynamic, heterogeneous networks [5], [6], [7], [8]. These challenges are aggravated by the mismatch between the classical byzantine adversary model and real-world blockchain environments, where most failures are non-malicious and probabilistic, such as hardware aging, transient communication noise, memory errors, and network congestion. Though temporary and unpredictable, such failures can severely degrade consensus stability, yet PBFT variants lack adaptive mechanisms to detect and exclude unstable nodes.

Previous research has proposed several approaches to address the scalability and fault tolerance of PBFT, including leader-committee methods (e.g., Gosig, LinBFT [9], [10]), multilayer architectures (e.g., PoBT [11]), scoring-based mechanisms (e.g., T-PBFT, SG-PBFT [12], [13]), and division-based methods (e.g., CBS-PBFT [14]). While these approaches reduce message overhead or improve fault handling, they remain tied to the Byzantine adversary model and fail to effectively address the probabilistic, non-malicious faults that dominate real-world networks.

To overcome these limitations, this study proposes Reliable Practical Byzantine Fault Tolerance (rPBFT), a consensus mechanism optimized for environments characterized by probabilistic and non-malicious faults. At its core, rPBFT introduces an adaptive Fault State Information (FSI) system that

Received 7 November 2024; revised 29 November 2025; accepted 7 December 2025. Date of publication 15 December 2025; date of current version 12 May 2026. This work was supported in part by the Ministry of Trade, Industry and Energy (MOTIE) through Training Industrial Security Specialist for High-Tech Industry under Grant RS-2024 00415520 supervised by the Korea Institute for Advancement of Technology, in part by the Ministry of Science and ICT (MSIT) through ICAN (ICT Challenge and Advanced Network of HRD) program under Grant IITP-2022-RS 2022-00156310, in part by the National Research Foundation of Korea under Grant RS-2025-00518150, and in part by Blockchain Privacy Preserving Techniques based on data encryption under Grant 2021-0-00518 that have been supervised by the Institute of Information & Communication Technology Planning & Evaluation (IITP). Recommended for acceptance by H. Kitagawa. (Eunyoung Lee and Namryeong Kim contributed equally to this work.) (Corresponding author: Ilgu Lee.)

Eunyoung Lee is with KB Life Partners, Seoul 04516, Republic of Korea (e-mail: eunyoung.lee@kblifepartners.co.kr).

Namryeong Kim and Chaerim Han are with the Department of Convergence Security Engineering, Sungshin Women's University, Seoul 06253, Republic of Korea (e-mail: namyoung0718@gmail.com; hcl7524@gmail.com).

Nayeon Shin is with Tndsoft, Seoul 13106, Republic of Korea (e-mail: nyshin@tndsoft.com).

Ilgu Lee is with the Department of Convergence Security Engineering, Sungshin Women's University, Seoul 02844, Republic of Korea (e-mail: iglee@sungshin.ac.kr).

Digital Object Identifier 10.1109/TBDATA.2025.3644587

continuously evaluates node reliability through message validity and latency, applying decay, penalty, and reward factors to reflect dynamic behavior. Based on this reliability assessment, each node independently forms a deterministic, leaderless committee by selecting highly reliable peers, reducing message complexity from $O(n^2)$ to $O(c^2)$ and eliminating single points of failure. Committees are periodically reconfigured to adapt to network changes, while overlapping participation preserves decentralization and enables parallel consensus execution. Experimental results indicate that rPBFT significantly improves latency, throughput, and consensus success rates compared to PBFT and HotStuff in fault-prone environments, while maintaining scalability and stability under diverse network conditions.

The main contributions of this study are as follows:

- We propose a reliability-aware, leaderless consensus mechanism that reduces message complexity and enhances fault resilience.
- Through extensive evaluation, we demonstrate that rPBFT achieves up to 70% lower latency and 90% higher throughput compared to PBFT under realistic fault models
- We present a practical framework that maintains decentralization and scalability even in heterogeneous, failure-prone networks.

The remainder of this paper is organized as follows: Section II reviews PBFT fundamentals, Section III analyzes previous improvement strategies, Section IV describes the proposed rPBFT mechanism, Section V presents its security properties and potential attack vectors, Section VI evaluates its performance and fault tolerance, and Section VII concludes the paper.

II. BACKGROUND

This section explains the basic concepts of PBFT.

A. PBFT Overview

PBFT is a consensus algorithm designed for asynchronous distributed networks that can tolerate Byzantine faults, including arbitrary or malicious behavior, without relying on a trusted third party (TTP) [3], [8]. A PBFT system with $3f+1$ nodes can reach consensus even if up to f nodes fail [8].

The Fischer–Lynch–Paterson (FLP) impossibility theorem states that, in an asynchronous network, both liveness (the ability to eventually reach consensus and safety (ensuring that all correct nodes decide on the same value) cannot be guaranteed simultaneously [15]. PBFT mitigates this limitation by prioritizing safety over liveness, assuming message delays do not grow indefinitely.

Compared with Nakamoto-style consensus algorithms, such as Proof of Work (PoW) and Proof of Stake (PoS), PBFT provides high throughput (TPS), immediate and absolute finality, and low power consumption. However, it suffers from weaker scalability and lower fault tolerance because of its quadratic message complexity $O(n^2)$ [16], [17], [18].

Table I summarizes the key characteristics of these consensus mechanisms. PoW and PoS can scale effectively and tolerate up to 50 percent faulty nodes but provide only probabilistic finality and require significant energy consumption. Conversely, PBFT

TABLE I
COMPARISON OF CONSENSUS ALGORITHMS

Consensus Algorithm	PoW	PoS	PBFT
Fault Tolerance	50%	50%	33%
TPS	<100	<1000	<2000
Finality	Probabilistic	Probabilistic	Absolute
Scalability	Strong	Strong	Weak
Power Consumption	High	Low	Low

PoW: proof of work; PoS: proof of stake; TPS: transaction per second

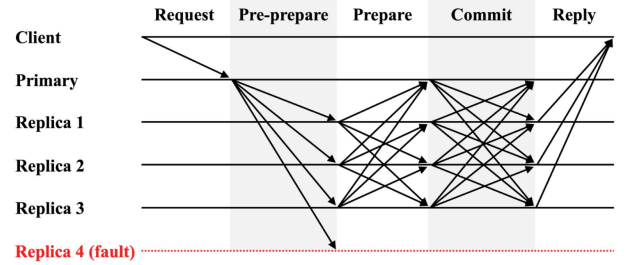


Fig. 1. Mechanism of PBFT.

guarantees strong consistency with low energy use but struggles to scale to large public networks because message complexity increases rapidly as the network grows.

B. PBFT Mechanism

PBFT nodes assume four roles: client, replica, primary, and backup. The client initiates consensus by sending a request message. Replicas maintain ledger copies, and among them one node is selected as the primary while the others act as backups. Fig. 1 illustrates the PBFT mechanism, which operates through five phases.

1) *Request. Client Sends a Request Message to the Primary Node:* In the request phase, the client sends a request message to the primary node. The client selects the primary node using (1):

$$p = v \bmod |n|, \quad (1)$$

where the number of primary nodes (p) is calculated by the modulo of the view number (v) and the total number of replicas (n). For instance, if $v = 12$ and $n = 10$, node two is designated as the primary node. The client transmits a request message $\langle \text{REQUEST}, o, t, c \rangle_{\delta_c}$ to the primary node, where o denotes the requested operation, t indicates the timestamp, c is the client, and δ_c represents the signature of the client. The timestamp t should be higher than that of the previously issued request message. The primary node starts the preprepare phase after receiving a request message from the client.

2) *Preprepare. Primary Node Multicasts Preprepare Messages to the Backup Nodes:* Upon receiving a request message from the client, the primary node immediately initiates the protocol unless the number of requests currently in progress exceeds a specified maximum value. The primary node logs the preprepare message $\langle \text{PRE_PREPARE}, v, n, d \rangle_{\delta_p, m}$ and sends it to backup nodes, where the view number (v) is a

unit of consensus. Consecutive numbers are assigned to v , and the sequence number (n) corresponds to the number of blocks progressing through consensus, n is the current block height + 1, and digest (d) is a hash value of the request message transmitted by the client. Backup nodes verify the pre-prepared message; if accepted, the algorithm proceeds to the prepare phase, otherwise no action is taken.

3) *Prepare. Backup Nodes Multicast the Prepare Message to Another Node:* During the prepare phase, backup nodes log the prepare message $\langle \text{PREPARE}, v, n, d, i \rangle_{\delta_i}$, where i identifies the sending node, and send it to all other nodes except themselves. Nodes receiving the prepared message check δ_i , v , and n to verify the message. When a node accepts one prepare message and $2f$ prepare messages, it changes to the prepare state and begins the commit phase.

4) *Commit. Replicas Multicast the Commit Message to Another Node:* In the commit phase, all replicas multicast a commit message to other replica. After logging the commit message, replica i transmits a commit message $\langle \text{COMMIT}, v, n, D(m), i \rangle_{\delta_i}$ to all nodes except itself. When $f + 1$ faultless replicas are in the prepare state, they transition to the commit state. In this case, the commit-local state is activated when $2f + 1$ commit messages are accepted. Nodes in the commit-local state then initiate the reply phase.

5) *Reply. Replicas Send a Reply Message to the Client:* In the reply phase, replicas execute the requested operation sent by the client and send the result to the client in a reply message $\langle \text{REPLY}, v, t, c, i, r \rangle_{\delta_i}$. The client accepts the result upon receiving $f + 1$ identical messages. If sufficient messages have not been received from the replica, it broadcasts a request message to all replica nodes, prompting retransmission if the request was already processed. Otherwise, the message is relayed to the primary node. If this node fails to multicast the request message, it indicates that the replica has several faults exceeding the acceptable limit, potentially triggering a view change.

III. RELATED WORK

This section analyzes research aimed at improving Practical Byzantine Fault Tolerance (PBFT), particularly addressing the trilemma of decentralization, scalability, and security [5]. Scalability refers to the ability of a network to maintain stability as its size grows, security concerns the establishment of a trustworthy shared ledger among all participants, and decentralization describes network operation without centralized control.

To understand recent developments, we analyzed PBFT studies published over the past five years and classified them into four categories: leader-committee, multilayer, scoring, and division. The leader-committee approach focuses on electing leader nodes or forming committees to drive consensus. The multilayer approach organizes nodes hierarchically by functionality or roles to reduce communication costs. The scoring approach evaluates, and filters nodes based on reliability metrics. The division approach splits nodes or data into subgroups to alleviate the burden of consensus. The following subsections review these categories in detail.

A. Leader-Committee-Based PBFT

The leader-committee approach selects a leader node or a set of leader nodes to manage consensus on behalf of the network. Some methods select a single leader node to coordinate consensus, while others divide the network into groups and appoint a leader for each group to form a leader committee. By involving only leader nodes in consensus, this approach reduces communication overhead in large networks.

Protocols such as Gosig and LinBFT employ verifiable random functions (VRFs) to randomly select leader nodes and rotate them for each block [9], [10]. The Stellar Consensus Protocol (SCP) combines Proof of Work (PoW) and BFT but faces challenges in maintaining efficiency and accuracy during final committee selection. Improved SCP (ISCP) addresses these issues by dividing nodes into multiple subcommittees based on PoW computations and running consensus in parallel within subcommittees [19]. Proteus and other consistent BFT protocols achieve stable performance in large networks by forming a root committee of trusted nodes, replacing it through a view-change mechanism if consensus fails [20], [21]. HotStuff advanced this line of research by introducing pipelining to reduce communication complexity to linear and improve throughput under partial synchrony [22]. HotStuff v2 further optimized leader rotation and responsiveness to mitigate latency bottlenecks [23]. Despite these improvements, leader-based approaches remain vulnerable to performance degradation or delays when leader nodes fail or experience network issues.

B. Multilayered PBFT

The multilayer approach organizes nodes into layers or functional groups to reduce time delays and performance degradation caused by network expansion. Layers can be defined based on device roles or capabilities, enabling different processing responsibilities.

For example, one security framework divides the network into infrastructure, core, and high-level layers [24]. The infrastructure layer includes IoT devices, the core layer comprises cluster heads and synchronization nodes, and the high-level layer consists of base stations with greater authority. This architecture improves scalability but introduces security risks if high-level nodes are compromised. The Proof of Block and Trade (PoBT) algorithm separates transaction verification and consensus formation [11]. Only nodes validated through smart contracts participate in consensus, reducing overhead while preserving security, though participant verification introduces additional cost. Li et al. [25] proposed a multilayer PBFT that partitions network members into smaller groups and layers to reduce message complexity. However, its performance is sensitive to malicious nodes owing to limited first-layer size and overlapping group structures. Overall, multilayer designs improve scalability but often weaken security and incur extra verification overhead.

C. Score-Based PBFT

The scoring approach evaluates node reliability to optimize participation in consensus. Nodes are assigned trust scores based

on factors such as latency, transmission speed, or response behavior. High-scoring nodes are prioritized or incentivized, while low-scoring nodes may be excluded to improve performance and security.

Several studies group nodes according to trust thresholds. T-PBFT [26], trust-PBFT [12], and reputational capital models [27] detect malicious nodes using score-based thresholds. The Blockchain Trust Model (BTM) assesses node trust using state, latency, and response metrics, classifying nodes as malicious if they exceed predefined thresholds [28]. SG-PBFT assigns each node an initial score of 100, rewarding correct behavior and penalizing deviations [28]. Other models, such as CertChain and blockchain-based trust management, exclude low-trust nodes and grant representative authority to highly trusted ones [29], [30]. However, these approaches often remove low-scoring nodes permanently, preventing unintentionally faulty nodes from regaining trust. While incentivizing reliable nodes can improve security, scoring systems alone struggle to resolve latency issues in large networks, and deposit-based penalties may exclude recovering nodes.

D. Division-Based PBFT

The division approach improves scalability by splitting nodes or data into subgroups or clusters, allowing parallel consensus. Nodes may be grouped by communication density, functional similarity, or predefined clusters, while data can be divided into shards or blocks.

For instance, Rashid et al. [24] applied evolutionary computation techniques such as genetic algorithms (GA) and particle swarm optimization (PSO) to cluster nodes. CBS-PBFT divides nodes into four clusters with specialized roles, including client, cluster primary, local primary, and backup [14], thereby reducing communication cost through linear messaging but weakening security owing to simplified leader selection. Li [31] grouped nodes based on communication density and function similarity but faced risks when nodes temporarily scored low owing to faults rather than malicious behavior. FastChain organizes Internet-of-Battlefield-Things (IoBT) devices to simulate military scenarios by considering computing and transmission capabilities [32]. Network-coded PBFT applies network coding to sharding to reduce bandwidth [33], though transmitting original data during prepare and commit phases can increase latency. While division-based designs aim to enhance scalability and reduce communication overhead, they often experience high latency as cluster or data size grows and may compromise security due to smaller consensus groups.

IV. PROPOSED METHOD

This study introduces Reliable Practical Byzantine Fault Tolerance (rPBFT), a consensus mechanism designed to detect and respond to faulty nodes in a distributed system without relying on a trusted third party (TTP). In this approach, faults are defined as non-malicious malfunctions caused by factors such as hardware defects, device aging, or transient network issues that may occur during the device or software deployment or operation. Byzantine nodes that intentionally behave maliciously

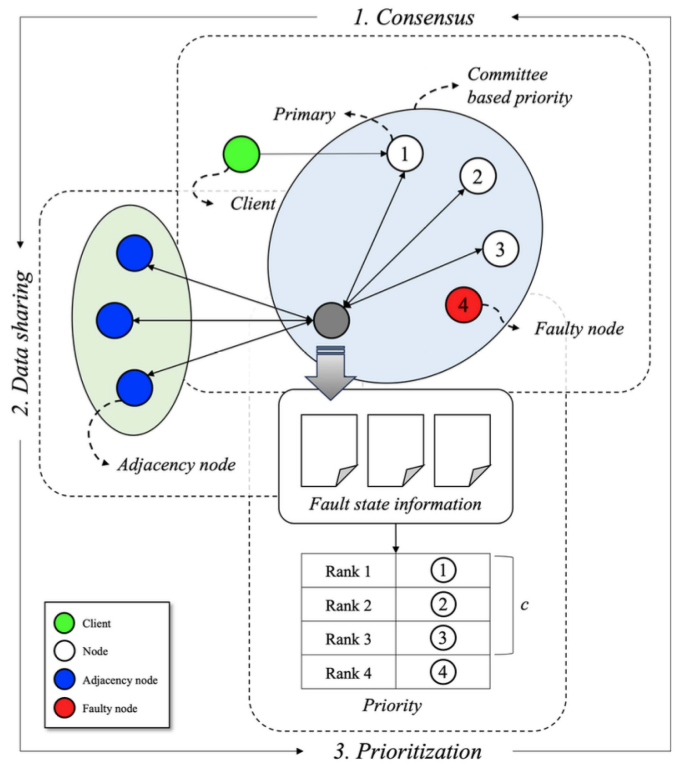


Fig. 2. Architecture of the rPBFT mechanism.

are not considered within the threat model. This distinction differentiates rPBFT from conventional BFT algorithms, which primarily focus on adversarial attacks. Instead, rPBFT aims to optimize performance in environments dominated by natural and probabilistic faults.

A. Overview

The proposed rPBFT mechanism is a consensus protocol specifically designed to maintain robustness in the presence of non-malicious faults. Unlike traditional PBFT, which concentrates on protecting against Byzantine adversaries, rPBFT prioritizes reliability and latency reduction under probabilistic fault conditions. At the core of rPBFT lies the Fault State Information (FSI) system, which continuously evaluates the reliability of each node based on its historical behavior. The overall procedure consists of three main stages: 1) fault information generation, 2) data sharing and priority calculation, and 3) committee-based consensus formation.

Fig. 2 presents the overall architecture of rPBFT, illustrating the generation, sharing, and utilization of FSI. During consensus, each node records message delivery validity and response delay to update its FSI. The FSI value gradually decays over time and is adjusted dynamically according to penalty and reward parameters: penalties are applied for abnormal or delayed behavior and rewards are given for correct and timely participation. This adaptive scoring mechanism allows long-term active nodes to have their reliability scores naturally reset, ensuring fair competition with newly participating nodes and preventing permanent score accumulation.

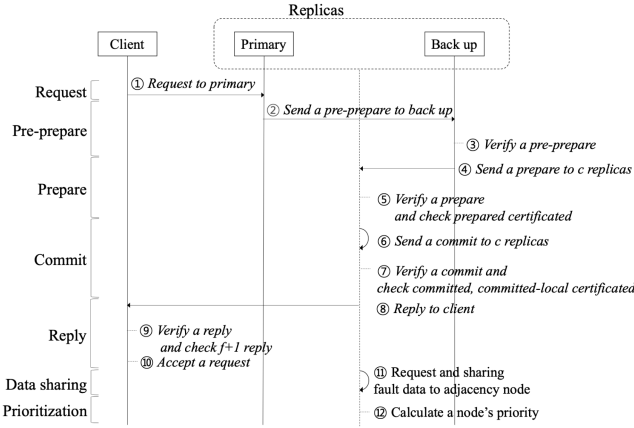


Fig. 3. Flow chart of rPBFT.

In the priority calculation stage, each node exchanges its FSI with neighboring nodes and aggregates the received information to compute a trust-based priority score. Using this score, nodes apply a deterministic randomization process based on identical global parameters such as block height, round number, and the previous commit hash. This ensures that all honest nodes independently derive an identical, reproducible ordering of nodes and select the top c nodes as the consensus committee. This committee-driven structure replaces the traditional single-leader model, enabling the network to maintain consensus progress even when individual nodes experience delays or degradation.

Fig. 3 illustrates the operation of the proposed method according to the roles of nodes in the network. Nodes are classified as clients, primary nodes, or backup nodes, and the consensus steps resemble those for PBFT. The process flow is as follows: 1) The client sends a request message to the primary node; 2) The primary node verifies the request message and sends a preprepared message to the backup nodes; 3) The backup nodes verify the preprepare message and transmit it to the top c nodes based on priority; 4) The nodes receiving the prepared message verify it and send a commit message to the c nodes with higher priority levels; 5) Once a node enters the committed local state, it sends a reply to the client; and 6) when the client receives $f + 1$ reply messages, it accepts the request. During prioritization, each replica requests and shares FSI obtained from adjacent nodes, combining it with its own FSI. These priorities are then applied in subsequent consensus formation. During this process, each node continuously updates its FSI at every consensus phase, sharing it with neighboring nodes to readjust priorities. The recalculated priority directly influences both committee selection and the subsequent consensus formation phase.

Fig. 4 shows the committee-based consensus phase, where all nodes deterministically derive the same committee and execute consensus within that selected subset. By adopting this committee-driven model, rPBFT reduces the message complexity of PBFT from $O(n^2)$ to $O(c^2)$, significantly lowering communication overhead. It also mitigates delays caused by leader replacement or network congestion, as no single node serves as a performance bottleneck. Consequently, rPBFT achieves high consensus success rates and throughput even in non-malicious

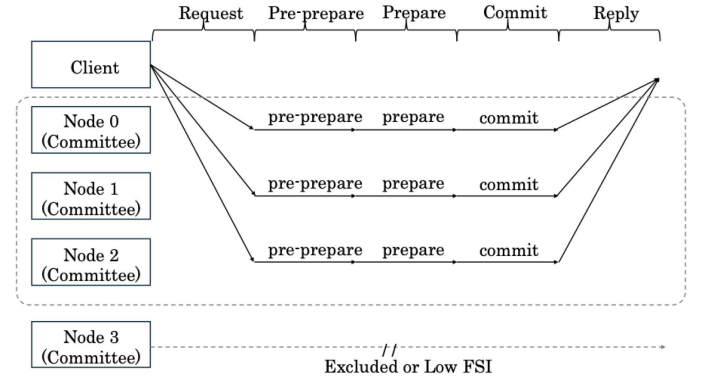


Fig. 4. Message flow of the rPBFT consensus process.

TABLE II
STRUCTURE OF EXAMPLE FSI

Block number	View number	Fault State
2	1	[0,1,1,0,0],[0,1,1,1,0],[0,1,1,0,0]
3	1	[0,1,1,0,0],[0,1,1,1,0],[0,1,1,0,0]
3	2	[0,1,1,0,0],[0,1,1,1,0],[0,1,1,0,0]
4	1	[0,1,1,0,0],[0,1,1,1,0],[0,1,1,0,0]

fault environments, while maintaining resilience against latency fluctuations and node performance degradation through adaptive FSI-based prioritization and committee selection.

B. Fault State Information Generation and Sharing

Each node records all messages generated during the consensus formation process and constructs FSI based on these records. FSI represents a quantitative measure of a node's reliability and comprises three elements: block number, view number, and a fault state vector.

Table II presents the details of an example FSI. For instance, if Node 1 receives messages from Nodes 2, 3, 4, and 5, it checks whether each message was successfully received. A successful reception is marked as '1,' while a failure as '0,' resulting in a binary vector such as [0, 0, 1, 0, 1]. Each node updates this reliability vector by aggregating the intersection of messages from the prepare and commit phases, associating the resulting vector with both block and view numbers for temporal traceability.

FSI is not merely a static record but a dynamically evolving reliability scores that decays over time. Each node periodically updates its FSI according to the following equation:

$$FSI_{(t+1)}(i) = \alpha \times FSI_t(i) - \beta \times penalty(i) + \gamma \times reward(i), \quad (2)$$

Here, α represents the decay factor ($0 < \alpha < 1$), determining the retention of past scores; β is a penalty weight applied to abnormal behavior; and γ is a reward weight for successful participation in consensus. Consequently, nodes that consistently operate normally experience a gradual decrease and periodic

Algorithm 1: FSI Construction and Update.

Require: *block_num, view_num, received_messages, total_nodes*
Ensure: Updated FSI entry

```

1: procedure CONSTRUCT_FSI(block_num, view_num, received_messages, total_nodes)
2:   prepare_vector, commit_vector, aggregate_vector
    $\leftarrow [0] \times \text{total\_nodes}$ 
3:   ForAll msg in received_messages.prepare
4:     if msg.is_valid() then
5:       prepare_vector[msg.sender]  $\leftarrow 1$ 
6:     end if
7:   end for
8:   ForAll msg in received_messages.commit
9:     if msg.is_valid() then
10:      commit_vector[msg.sender]  $\leftarrow 1$ 
11:    end if
12:   end for
13:   for i  $\leftarrow 0$  to total_nodes - 1 do
14:     aggregate_vector[i]  $\leftarrow$ 
       (prepare_vector[i]  $\wedge$  commit_vector[i])
15:   end for
16:   fsi_entry  $\leftarrow$  {block_number :
     block_num, view_number :
     view_num, fault_state :
     [prepare_vector, commit_vector, aggregate_vector]}
17:   FSI_history.append(fsi_entry)
18:   return fsi_entry
19: end procedure

```

reinitialization of their FSI values. In contrast, nodes that frequently exhibit faulty behavior accumulate penalties, resulting in lower FSI values and reduced reliability.

The generated FSI is selectively shared among adjacent nodes to reduce observational bias and mitigate the influence of individual errors on the overall reliability assessment. Each node stores its own FSI (denoted as A_s) in the first position and sequentially appends FSIs received from neighboring nodes ($A_1, A_2, \dots, A_k$). For example, if Node 1 shares its FSI with Node 2, Node 2 can refine its reliability estimate using Node 1's observation. Collecting FSI values from multiple peers enhances fault prediction accuracy while maintaining minimal communication overhead. Algorithm 1 presents the pseudocode for the FSI generation phase.

Fig. 5 illustrates the FSI sharing structure among adjacent nodes. Each node generates its FSI based on message receptions during the prepare and commit phases and exchanges this information with its neighbors to perform multidimensional trust estimation. This partial sharing model prevents redundant message flooding across the network, maintaining low communication complexity while enabling localized reliability inference.

By integrating dynamically decaying reliability evaluation with adjacency-based FSI sharing, the proposed rPBFT mechanism effectively identifies naturally faulty nodes and reduces

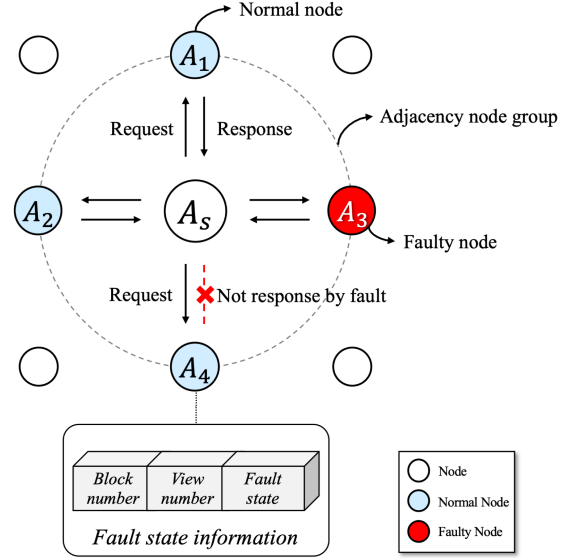


Fig. 5. Sharing FSI from adjacency nodes.

consensus failure rates. Nodes with higher fault likelihoods are automatically assigned lower priorities and excluded from committee formation, while stable nodes are selected as core participants. Consequently, rPBFT achieves reduced message complexity—from $O(n^2)$ to $O(c^2)$ —and maintains high consensus success rates and throughput even under non-malicious fault conditions.

C. Prioritization

Each node computes a priority score for all peers based on its locally maintained FSI. This score quantitatively reflects the reliability of each node and determines how frequently the node participates in consensus. The purpose of prioritization is to identify reliable nodes and proactively exclude nodes with a high likelihood of faults, thereby reducing latency and avoiding unnecessary view changes. Algorithm 2 presents the pseudocode for this procedure.

The priority score is calculated as a sliding accumulation of FSI values from the most recent k blocks or rounds with a decay mechanism to reduce the impact of outdated data:

$$\text{priority}(i) = \sum_{t=n-k}^n FSI_t(i), \quad (3)$$

A higher value indicates consistent, successful participation in recent consensus rounds. For example, given three observed fault vectors $[0,1,0,1,0]$, $[0,1,1,1,1]$, and $[0,1,0,0,1]$, the summed vector becomes $[0,3,1,2,2]$, indicating that Node 2 is the most reliable. Nodes with high cumulative scores are selected more frequently for consensus participation, while those with low scores are deprioritized.

The decay factor α plays a critical role in balancing responsiveness to new faults and the stability of accumulated trust. When α is close to 1 (e.g., $\alpha \geq 0.9$), past reliability is preserved for a long time, making it difficult for new nodes to gain priority

Algorithm 2: Priority Calculation Based on FSI.

Require: $node_id, FSI_history, current_block, k \triangleright k$: number of recent blocks
Ensure: $Priority(i)$ – priority value for node i

- 1: **procedure** CALCULATE_PRIORITY($node_id, FSI_history, current_block, k$)
- 2: $priority \leftarrow 0$
- 3: **// Get k most recent blocks**
- 4: $recent_blocks \leftarrow get_recent_blocks(FSI_history, current_block, k)$
- 5: **// Sum FSI values from recent k blocks**
- 6: **ForAll** $block$ **in** $recent_blocks$
- 7: $fsi_entry \leftarrow get_FSI_entry(block, node_id)$
- 8: **if** fsi_entry **exists** **then**
- 9: **// Use aggregate vector (intersection of prepare & commit)**
- 10: $aggregate_vector \leftarrow fsi_entry.fault_state[2]$
- 11: **// Sum successful participations**
- 12: **ForAll** $value$ **in** $aggregate_vector$
- 13: $priority \leftarrow priority + value$
- 14: **end for**
- 15: **end if**
- 16: **end for**
- 17: **return** $priority$
- 18: **end procedure**

and potentially introducing long-term bias. Conversely, a very small α ($\alpha \leq 0.5$) causes the system to forget history too quickly, leading to unstable committee selection. Through experimental tuning, this study recommends setting α in the range of 0.7–0.85 to maintain stability while allowing new nodes to join over time.

The penalty (β) and reward (γ) weights adjust how strongly abnormal or successful behaviors influence FSI. A larger β quickly reduces the score of unreliable nodes, allowing faster exclusion but possibly overreacting to transient noise. A higher γ accelerates score recovery for stable nodes but can result in persistent long-term advantages. Empirically, values of $\beta=0.1$ –0.3 and $\gamma=0.5$ –0.8 provide balanced responsiveness and stability in fault-prone environments.

When a node lacks sufficient FSI (e.g., a newly joined node), rPBFT falls back to conventional PBFT behavior to preserve liveness and ensure that the network remains operational even under incomplete trust data.

D. Composition of Committee

Algorithm 3 presents the pseudocode for the committee formation process. After each node finishes computing reliability scores from FSI, it independently forms a consensus committee by selecting the top c nodes with the highest priority values. Consensus messages are then exchanged primarily within this committee, reducing communication complexity from $O(n^2)$ in conventional PBFT to $O(c^2)$. This reduction significantly improves efficiency and lower latency.

Committee selection is deterministic across the entire network. All nodes use identical shared input values—such as the

Algorithm 3: Committee Formation Based on Priority.

Require: $node_id, all_nodes, FSI_history, current_block, k, c \triangleright c$: committee size
Ensure: Local committee for this node

- 1: **procedure** FORM_LOCAL_COMMITTEE($node_id, all_nodes, FSI_history, current_block, k, c$)
- 2: **// Step 1: calculate priority for all nodes from this node's perspective**
- 3: $node_priorities \leftarrow []$
- 4: **for each** $candidate_id \in all_nodes$ **do**
- 5: $p \leftarrow CALCULATE_PRIORITY(candidate_id, FSI_history, current_block, k)$
- 6: $d \leftarrow NETWORKDISTANCE(node_id, candidate_id)$
- 7: $node_priorities.APPEND(\langle candidate_id, p, d \rangle)$
- 8: **end for**
- 9: **// Step 2: sort by priority (desc), then distance (asc)**
- 10: $node_priorities \leftarrow SORT(node_priorities, by (-priority, +distance))$
- 11: **// Step 3: select top c nodes**
- 12: $local_committee \leftarrow TOPC(node_priorities, c)$
- 13: **// Step 4: ensure self is included**
- 14: **if** $node_id \notin local_committee$ **then**
- 15: $REPLACELOWESTPRIORITY(local_committee, node_id)$
- 16: **end if**
- 17: **return** $local_committee$
- 18: **end procedure**

current block height, view number, and the previous commit hash—to seed the committee calculation. This guarantees that all honest nodes derive the same committee composition, even under asynchronous conditions or in the presence of message delays and network forks. Deterministic construction prevents divergence of committee views and supports a stable global consensus configuration.

To maintain fairness and prevent long-term dominance by a small set of nodes, a decay mechanism is applied to FSI-based priorities. Over time, each node's reliability score gradually decreases, reducing overreliance on historical performance. Long-term stable nodes periodically have their scores renormalized, allowing new nodes to enter the committee. Conversely, nodes that frequently show faults or delayed responses accumulate penalties that outweigh the decay effect, making their selection less likely.

Although each node executes the committee selection locally, the use of identical seeds and procedures guarantees global consistency of the committee view. As FSI values are updated in every consensus round, the committee composition adapts dynamically to the current state of the network. This dynamic adaptation enables the system to maintain high stability and fault tolerance even in environments with fluctuating latency and variable node performance.

Fig. 6 illustrates how each node simultaneously acts as the leader of its own committee while also serving as a member of other committees. In practice, each node transmits consensus

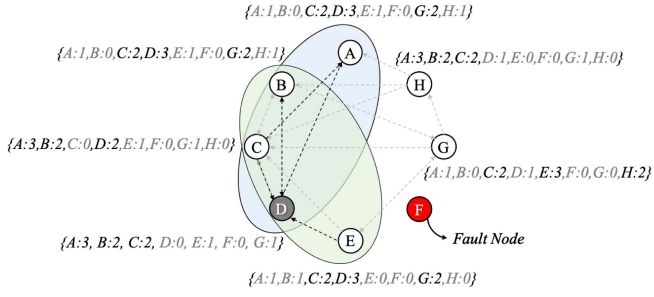


Fig. 6. Organization of a committee by a node.

TABLE III
RECOMMENDED PARAMETER SETTINGS FOR COMMITTEE FORMATION

Network size (N)	Committee size (c)	α	Design note
50–200	8–12	0.75–0.85	Fast adaptation with moderate history retention
200–1000	10–20	0.70–0.85	Balanced security and communication cost
1000–2000	15–25	0.70–0.80	More stability needed with larger networks
>2000	20–30	0.70–0.80	Larger committee to maintain safety under high fault rates

messages to its selected c high-priority peers and, in parallel, responds to committees where it has been chosen as a member. This dual role improves redundancy, strengthens inter-node connectivity, and increases resilience against local node failures, while avoiding the single-leader bottleneck inherent in conventional PBFT.

The committee size c directly impacts both performance and security. A small c (e.g., $c < 5$) increases vulnerability because a single faulty node can disproportionately disrupt consensus, while a large c (e.g., $c > 30$) leads to excessive messaging and diminished scalability benefits. Experimental results indicate that $c = 10\text{--}20$ achieves the optimum balance between latency, throughput, and fault tolerance in networks up to 1000 nodes, while $c = 20\text{--}30$ may be preferable for very large networks. To support practical deployment, Table III provides recommended ranges of c and the FSI decay factor α for different network sizes.

As shown in Table III, small networks (50–200 nodes) benefit from fast adaptation with α around 0.75–0.85 and committee sizes of 8–12 nodes, while larger networks require slightly smaller decay factors ($\alpha = 0.70\text{--}0.80$) and larger committees (15–30 nodes) to maintain stability and fault tolerance. These parameter guidelines provide a practical reference for balancing latency, throughput, and security depending on network size, offering a concrete reference for practitioners deploying rPBFT.

E. Consensus Process

The consensus process in rPBFT builds upon the traditional PBFT protocol but is optimized to operate within a dynamically

formed committee structure. Each node independently determines its own consensus committee using priority values derived from FSI, ensuring that only nodes with high reliability actively participate in consensus. Once a node receives at least $\frac{c}{2} + 1$ valid responses from its committee peers, it sends a reply to the client, guaranteeing progress even when some committee members fail. When the committee size is c and the number of faulty nodes within the committee is b , the message complexity is reduced to $2(c - b)^2$, which is significantly lower than the conventional PBFT complexity of $2(n - f)^2$ [8]. This reduction in communication overhead directly improves latency, throughput, and scalability, especially in networks where $c \ll n$. As summarized in Table III, choosing $c = 10\text{--}20$ for medium-scale networks and $c = 20\text{--}30$ for large-scale systems yields an effective balance between security and performance.

Unlike traditional PBFT, which relies on a single leader node to coordinate consensus, rPBFT eliminates the need for a centralized leader. Each node autonomously derives its committee using local FSI values and shared deterministic seed parameters (block height, view number, and previous commit hash). This leaderless design removes the bottleneck and single point of failure inherent in leader-based models, significantly improving fault tolerance and system robustness under heterogeneous network conditions.

Furthermore, while most committee-based BFT protocols restrict message exchanges strictly within the selected committee, rPBFT allows partial message reception from nodes outside the committee. This design choice enables nodes to incorporate reliability information from other committees, thereby improving global fault awareness and increasing the resilience of consensus formation against localized performance degradation or communication noise. Consequently, the rPBFT consensus mechanism achieves two major improvements simultaneously: 1) Reduced message complexity from $O(n^2)$ to $O(c^2)$ while preserving safety and liveness. 2) Strong decentralization through a fully leaderless, adaptive committee structure.

Together, these enhancements allow rPBFT to maintain high consensus success rates, minimize retransmission costs, and deliver stable throughput even in large-scale networks subject to natural, non-malicious faults such as hardware aging, random network errors, or transient communication delays.

V. SECURITY ANALYSIS

The security of rPBFT is critical to its viability as a scalable and fault-tolerant consensus protocol for blockchain systems. This section extends the classical PBFT security analysis and incorporates the new elements introduced by rPBFT: dynamic committee formation and FSI-based prioritization.

A. Fault Tolerance and Byzantine Resistance

Classical PBFT guarantees both safety and liveness when the total number of nodes N satisfies the well-known bound: $N \geq 3f + 1$, where f is the maximum number of Byzantine (malicious) nodes. This implies that PBFT can tolerate up to $f = \lfloor \frac{N-1}{3} \rfloor$ faulty nodes, and consensus fails once more than one-third of the network becomes Byzantine.

In rPBFT, consensus is performed within committees of size c , dynamically selected based on FSI. To preserve safety and liveness, each committee must contain an honest majority. Formally, if a committee of size c contains b Byzantine nodes, the system remains safe when $c \geq 3b + 1$, or equivalently, when the fraction of Byzantine nodes within a committee satisfies $\frac{b}{c} < \frac{1}{3}$.

Because rPBFT prioritizes reliable nodes when forming committees, it reduces the probability that faulty or unstable nodes dominate the committee, allowing the protocol to maintain effective safety even if the global network fault ratio temporarily exceeds $\frac{1}{3}$. For instance, if a network with $N=100$ and 40 unstable nodes, conventional PBFT would break safety, but rPBFT can maintain consensus if most unreliable nodes have low FSI and are excluded from committees.

This adaptive selection also slows Sybil attacks. New or forged identities start with low priority because their FSI is near zero. As priority accumulates over time according to $priority_{t+1}(i) = \alpha \times priority_t(i) - \beta \times penalty(i) + \gamma \times reward(i)$, attackers must participate honestly for multiple rounds to achieve committee inclusion. This time and resource cost creates a strong barrier against rapid network infiltration.

B. Network Resilience

The FSI-driven committee adaptation in rPBFT strengthens resilience against several common network-level attacks:

1) *Denial-of-Service (DoS) or Latency Attacks*: Nodes under DoS or experiencing severe delays fail to respond in the prepare/commit phases, which directly reduces their FSI scores owing to accumulated penalties (β). Over successive rounds, these nodes drop in priority and are excluded from future committees. Thus, the network self-heals by migrating consensus participation toward unaffected, stable nodes.

2) *Sybil and Identity Flooding*: Sybil nodes must accumulate FSI to gain committee access. As FSI grows gradually through successful participation weighted by γ , attackers cannot instantly dominate committees. Consequently, the decay factor α ensures that old dormant Sybil nodes lose their advantage, requiring them to stay active over time.

3) *Eclipse or Partition Attacks*: Because FSI values are shared among neighboring nodes, it is difficult for an attacker to fully isolate a target. Even if some nodes are cut off, other nodes continue to share and update FSI, enabling committees to form with reliable peers outside the attacker's partition.

4) *Long-Range or History-Replay Attacks*: Unlike PoS systems, rPBFT continuously re-evaluates node reliability each round. Old reliability scores decay ($\alpha < 1$), preventing attackers from indefinitely exploiting past honest behavior. Periodic FSI updates force long-term attackers to remain active and reliable, raising the cost of sustained influence.

C. Committee Collusion

A potential risk in rPBFT is committee collusion, where a small set of high-priority nodes repeatedly dominate selection and may coordinate to influence consensus. While the decay mechanism already reduces long-term dominance, additional safeguards can further mitigate this threat: 1) Periodic priority

TABLE IV
EXPERIMENTAL ENVIRONMENT

Section	Specifications
CPU	10-core CPU Apple M2 Pro chip
RAM	16 GB
OS	macOS 14.1
Python	3.10.1

resets: Every k rounds, a fraction (e.g., 20%) of committee slots can be filled by nodes chosen randomly from the non-committee pool, disrupting long-term collusion. 2) Seed randomization: Committee seeds can combine global block hashes or verifiable random functions (VRF) with the standard deterministic seed to add unpredictability. 3) Participation caps: Nodes exceeding a threshold of consecutive committee selections can have their priority temporarily reduced to enforce diversity. These design choices increase unpredictability and reduce the risk of quasi-centralized control, while retaining the communication efficiency of deterministic committee formation.

Overall, rPBFT preserves the baseline PBFT safety bound ($N \geq 3f + 1$), while adapting committee membership to observed reliability. This approach strengthens fault tolerance in practical networks with natural, non-malicious failures and reduces the impact of targeted attacks such as DoS, Sybil, and partitioning, without compromising decentralization.

VI. EVALUATION AND ANALYSIS

A. Experimental Environment

This section evaluates the latency, throughput, and consensus success rates of PBFT and the proposed rPBFT. To reproduce a blockchain network environment, we implemented a Python-based simulator and tested the system under various fault-free and fault-prone conditions.

Table IV summarizes the experimental setup. All experiments were conducted on a 10-core Apple M2 Pro CPU, 16GB of RAM, and macOS 14.1. The simulator was developed in Python 3.10.1. Because PBFT has relatively few practical, optimized implementations, quantitative comparison with other BFT algorithms is limited; however, we focused on evaluating rPBFT versus PBFT under a range of network conditions.

The simulated network supported up to 1,000 nodes with a fully connected topology, meaning every node could directly communicate with others. To approximate real-world network dynamics, we introduced asynchronous communication with message delays following a normal distribution (mean = 50ms, standard deviation = 10ms). All nodes were assumed to have equal computational capability and maintained a full local copy of the blockchain and recent consensus messages. Nodes were randomly assigned as clients, primary nodes, or backup nodes at the start of each consensus round. During each round, nodes were marked as faulty according to a selected reliability model.

1) *Reliability Model*: To emulate real-world device behavior, this study used well-established reliability models that estimate device lifetime and failure probability [34]. These models allow analysis of how natural faults affect consensus

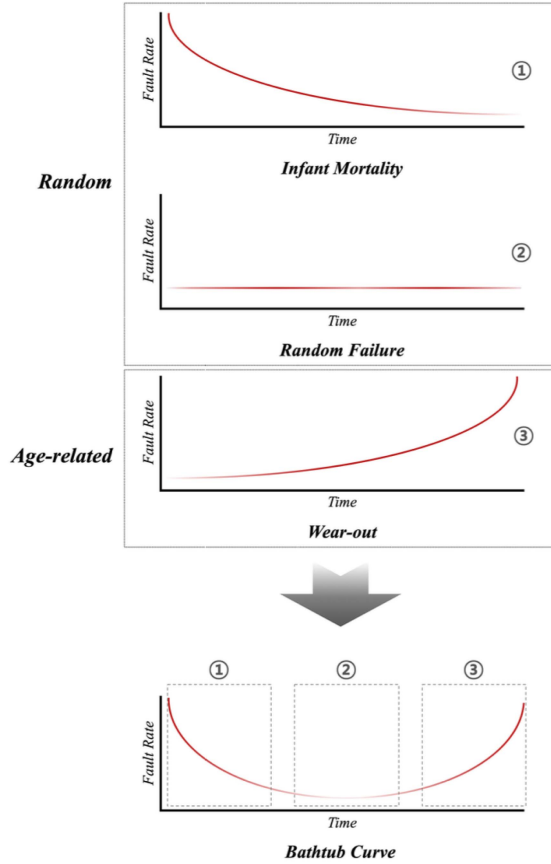


Fig. 7. Reliability models (infant mortality, random failure, wear-out, bathtub curve).

in distributed networks. The evaluation employed three classical models: wear-out, infant mortality, and random failure. These models represent failures caused by device aging, early-stage imperfections, and unpredictable random faults, respectively. The fault probability of each model over time is shown in Fig. 7.

The infant mortality model reflects the high failure rate that often occurs during the early stages of device or software deployment, which gradually stabilizes over time. The random failure model represents unpredictable failures that occur at a low and constant rate, regardless of device age. The wear-out model represents failures caused by hardware aging, where the fault rate increases as devices operate longer. These models were implemented using the Python reliability library: the infant mortality model using a Weibull distribution, the random failure model using an exponential distribution, and the wear-out model with a lognormal distribution. Each model generated fault rates ranging from 1 to 1,000 over time.

Additionally, a bathtub curve model was used to approximate realistic device lifecycle behavior. The bathtub curve combines three failure modes: infant mortality characteristics at the beginning, random failure in the middle, and wear-out at the end. For the experiment, this combined curve was randomly segmented and assigned to nodes, thereby reflecting heterogeneous reliability conditions like those found in practical large-scale networks.

2) *Evaluation Metrics*: Three evaluation metrics were selected: latency, throughput, and consensus success rate. Latency was defined as the time required to complete one round of consensus. For PBFT, it was measured from the moment a client transmitted a request message until the corresponding reply is received. For rPBFT, it is measured from the time the client sends a request to the completion of the prioritization and consensus process. Factors unrelated to the consensus algorithm, such as node-level communication delays and blockchain storage operations, were excluded.

Throughput refers to the amount of information successfully processed per second, calculated as:

$$\text{Throughput (bps)} = \frac{\text{Block Height} \times \text{Block Size}}{\text{Transmission Time}} \quad (4)$$

The total processed information was obtained by multiplying the height of the current block by the block size, and dividing by the total transmission time.

The consensus success rate indicates the percentage of successful consensus rounds out of the total attempts:

$$\text{Consensus Success Rate (\%)} = \frac{\text{Block Height}}{\text{Total Number of Consensus}} \quad (5)$$

3) *Evaluation Framework*: The simulator comprises “Experiment Framework,” “Reliability Model,” “PBFTHandler,” “Node,” “Consensus Algorithm,” and “Blockchain.” Fig. 8 illustrates the class diagram of the experiment framework. The experiment framework configured the number of nodes, transactions, faulty nodes, and the consensus algorithm. Once parameters were set, nodes were created and assigned unique IDs: the client was ID 0, the primary node was ID 1, and the remaining backup nodes were IDs from 2 to n . Faulty nodes were designated according to the selected reliability model and behaved abnormally based on the assigned fault probability.

The PBFTHandler implemented both PBFT and rPBFT. It managed asynchronous message exchanges in a fully connected network where each node could directly communicate with others. All consensus messages were recorded locally to verify duplication, validity, and counts.

To maintain data integrity, each node computed the hash (digest) of the client’s request message and compared it with the digest in received messages. This prevented tampering and ensured correctness. Once integrity was verified, rPBFT advanced to the next consensus stage based on PBFT’s validation rules, but without unnecessary waiting. Unlike PBFT, which delays progress until all backup nodes respond or a timeout occurs, rPBFT allows a committee of reliable nodes to proceed once the required number of valid messages is reached. This approach reduces latency and improves throughput by avoiding delays caused by faulty or slow nodes.

After consensus completes, the client verifies at least $f+1$ replies by checking the digest of the received messages against the original request. If verification succeeds, the new block is added to the blockchain; otherwise, the round is retried. This process repeats for the configured number of transactions,

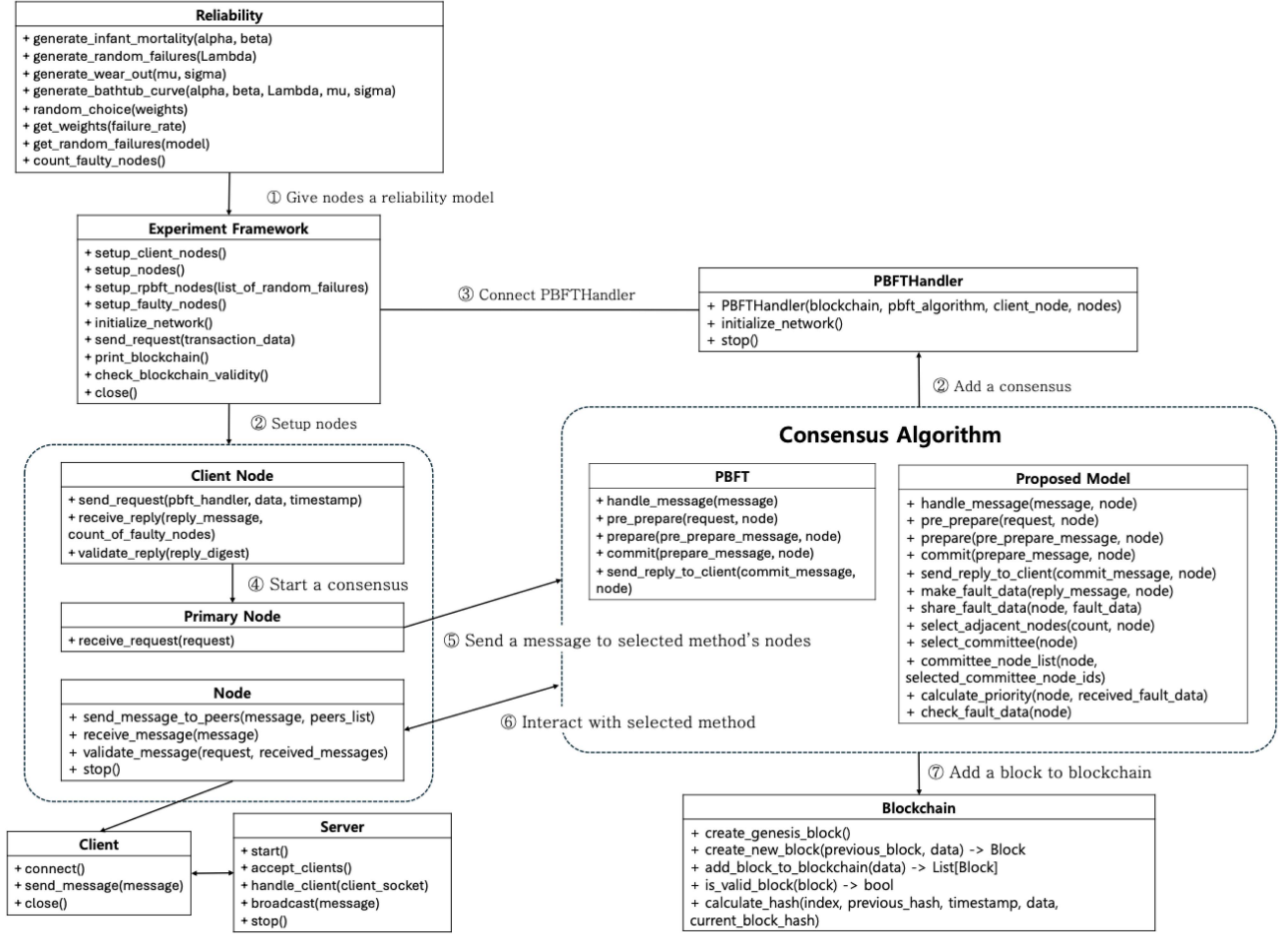


Fig. 8. Class diagram of the evaluation framework.

and the results are used to calculate throughput and consensus success rate.

B. Evaluation Results

1) *Evaluation Results Under the Reliability Model:* This experiment evaluated PBFT, HotStuff, and rPBFT under the bathtub curve reliability model, which includes Infant Mortality, Random Failures, and Wear-Out phases. One hundred nodes generated 1,000 blocks, and the test was repeated 100 times to obtain average results. Fig. 9 shows the consensus latency of each algorithm.

In the Infant Mortality phase, PBFT and HotStuff showed high latency (over 1.2 s) owing to leader synchronization delays and heavy message exchange, while rPBFT remained stable by excluding unreliable nodes through reliability-based committee selection. During the Random Failures phase, all algorithms stabilized at 0.2–0.3 s, but rPBFT exhibited the lowest latency variance because its FSI-based dynamic committees mitigated transient faults. In the Wear-Out phase, PBFT and HotStuff experienced sharp latency increases from node degradation and message congestion. rPBFT maintained stable latency by periodically updating FSI and excluding persistently faulty nodes. Overall, rPBFT achieved the lowest and most stable latency

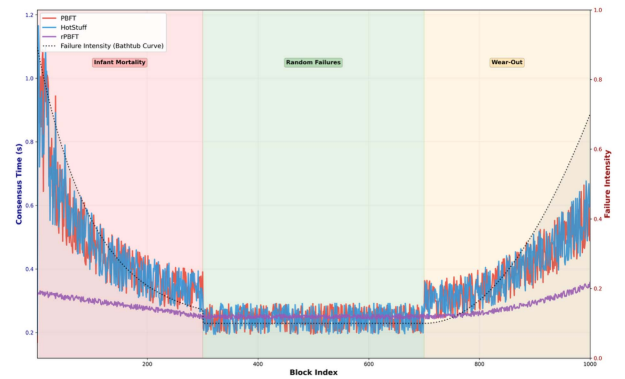


Fig. 9. Latency of PBFT, HotStuff and rPBFT for a reliability model.

across all phases, showing superior fault tolerance and scalability compared to PBFT and HotStuff.

2) *Evaluation Results of Performance Analysis:* This section evaluates PBFT, HotStuff, and rPBFT in both fault-free and fault-prone environments using latency, throughput, consensus success rate, message complexity, and resource usage. In the fault-free setting, all nodes operated normally. In the faulty setting, node failures followed the bathtub curve reliability model,

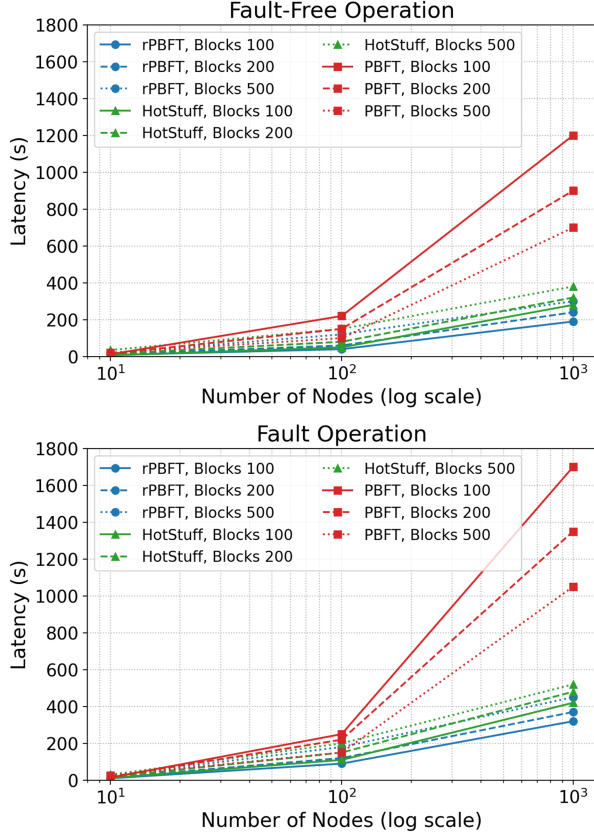


Fig. 10. Latency of PBFT, HotStuff and rPBFT in both the faulty and fault-free environments.

where early, mid-life, and late-stage nodes represented infant mortality, random failure, and wear-out, respectively. Networks ranged from 100 to 1,000 nodes and generated 100 to 1,000 blocks, with results averaged over 100 trials. Figs. 10 and 11 illustrate the latency and throughput trends.

In fault-free environments, all nodes operated normally without retransmissions, and all three algorithms slowed as the network size increased; however, scalability differed significantly. PBFT suffered from quadratic message complexity, causing latency to climb above 1,000 s and throughput to fall below 30 Mbps with 1,000 nodes, while HotStuff scaled better, maintaining latency in the few-hundred-second range and throughput near 55 Mbps. rPBFT achieved the most stable performance by reducing broadcast overhead through its committee-based consensus, maintaining latency at only a few hundred seconds and throughput around 65–70 Mbps even at the largest scale. In fault-prone environments, where node failures followed the bathtub curve model to reflect infant mortality, random failures, and wear-out phases, PBFT experienced severe retransmissions and frequent view changes that caused long delays and throughput collapse. HotStuff adapted better but still degraded under frequent faults. rPBFT remained the most stable by dynamically excluding unreliable nodes and selectively revalidating messages instead of restarting full consensus, reducing average latency by about 70% and improving throughput by nearly 90% compared to PBFT while consistently outperforming HotStuff.

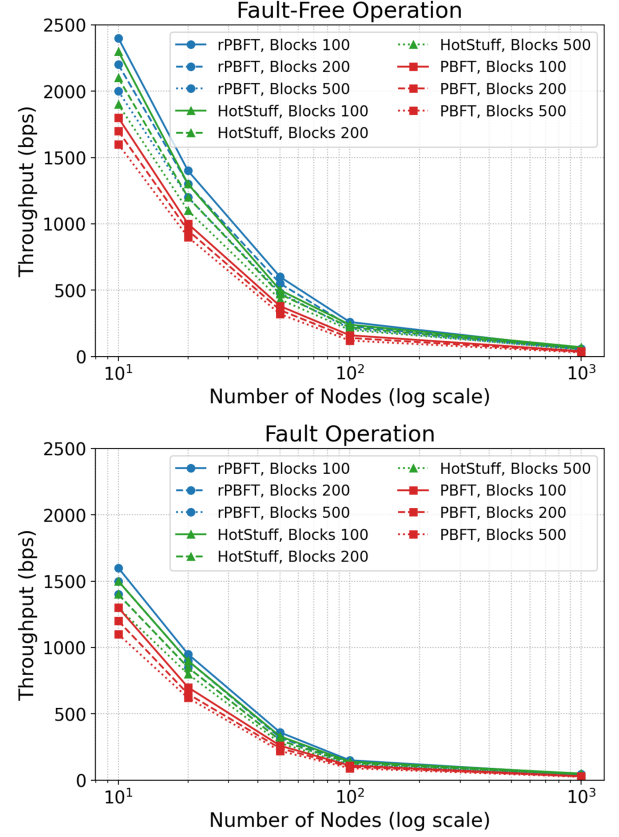


Fig. 11. Throughput of PBFT, HotStuff and rPBFT in both the faulty and fault-free environments.

TABLE V
MESSAGE COMPLEXITY COMPARISON IN FAULT-FREE AND FAULT OPERATION

Transactions	PBFT	Hotstuff	rPBFT
Fault-Free			
100	1,159,000	89,420	191,610
200	2,277,000	171,000	364,275
500	5,555,300	407,000	867,720
Fault			
100	6,226,600	277,200	596,655
200	12,925,700	480,300	1,019,910
500	29,332,200	1,450,000	3,160,080

These results confirm that rPBFT provides stronger scalability and fault tolerance than traditional PBFT and HotStuff, making it well-suited for large, heterogeneous blockchain networks.

Table V compares the message complexity of PBFT, HotStuff, and rPBFT in both fault-free and faulty environments. In a 1,000-node network with 500 transactions under fault-free conditions, PBFT required about 5.56 million messages, rPBFT about 0.87 million, and HotStuff about 0.41 million. HotStuff shows the lowest message complexity due to its leader-based, pipeline-oriented consensus, which minimizes communication steps. However, this simplicity can increase latency because performance depends heavily on a single leader; as shown in Figs. 10 and 11, HotStuff often responded more slowly than rPBFT in large or unstable networks. Thus, lower message complexity

TABLE VI
CPU, MEMORY, AND NETWORK USAGE COMPARISON IN FAULT-FREE AND
FAULT OPERATION

Consensus	PBFT	Hotstuff	rPBFT
Fault-Free			
CPU Usage (%)	0.347	0.459	0.539
Memory Usage (MB)	807.32	893.07	871.36
Network Usage (MB)	5,047.46	4,975.52	5,241.67
Fault			
CPU Usage (%)	1.751	0.785	0.886
Memory Usage (MB)	928.56	802.60	766.37
Network Usage (MB)	7,428.78	6,610.00	7,095.76

does not necessarily mean faster or more reliable consensus, highlighting a trade-off between communication simplicity and latency.

In fault-free conditions, rPBFT exchanges far fewer messages than PBFT overall, but it performs slightly more control communication than a plain broadcast system because it adds steps for committee formation and fault detection. These steps introduce moderate overhead but help prevent consensus failures and improve stability. The overhead can be reduced by adjusting the committee reconfiguration interval according to network stability—delaying reconfiguration when the network is stable to avoid redundant updates—and by batching or compressing verification messages using methods such as Merkle Trees or asynchronous delivery.

Under faulty conditions, the differences become clearer. PBFT continues broadcasting to all nodes, leading to exponential message growth as faults increase, reaching about 29.33 million messages at 1,000 nodes and 500 transactions. rPBFT limits this growth to about 3.16 million, while HotStuff stays at about 1.45 million. This indicates that rPBFT accepts a moderate amount of control traffic during normal operation but successfully prevents message explosion when faults occur, keeping complexity manageable while maintaining high reliability. Overall, rPBFT achieves a better balance between scalability and fault tolerance than PBFT, and it is more robust than HotStuff in large, heterogeneous blockchain networks where node failures are common.

Table VI compares the CPU, memory, and network usage of PBFT, HotStuff, and rPBFT in both fault-free and faulty environments using 30 nodes and 500 blocks. CPU usage was measured as the average percentage of total CPU consumed per second by each consensus process. In the fault-free environment, PBFT used 0.347%, HotStuff 0.459%, and rPBFT 0.539%, showing that rPBFT consumed slightly more CPU than PBFT because of its committee management and fault detection. In the faulty environment, PBFT's CPU usage increased to 1.751%, while HotStuff and rPBFT remained efficient at 0.785% and 0.886% respectively. This indicates that PBFT incurs high computational cost when faults occur, whereas HotStuff and rPBFT adapt more efficiently, with HotStuff remaining the most lightweight owing to its leader-based design, though at the expense of higher latency as shown in Figs. 10 and 11.

Memory usage represents the amount of RAM consumed during consensus execution. In the fault-free environment, PBFT used 807 MB, HotStuff 893 MB, and rPBFT 871 MB, reflecting the extra data structures for block linking in HotStuff and for

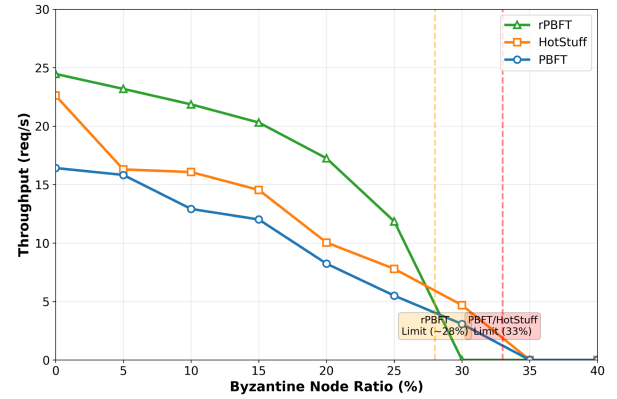


Fig. 12. Throughput variation according to Byzantine node ratio.

committee coordination in rPBFT. Under faulty conditions, PBFT's memory usage increased sharply to 928 MB, while HotStuff dropped to 802 MB and rPBFT to 766 MB, showing that the two adaptive protocols reduce redundant buffering and state replication when handling failures.

Network usage was calculated as the total transmitted and received traffic during consensus. In the fault-free environment, PBFT used 5,047 MB, HotStuff 4,975 MB, and rPBFT 5,241 MB; rPBFT generated slightly more traffic owing to committee selection and verification messages, while HotStuff minimized network load through its streamlined pipeline. In the faulty environment, PBFT's traffic increased to 7,428 MB, while HotStuff and rPBFT required 6,610 MB and 7,095 MB respectively, confirming that PBFT suffers from excessive broadcasting when faults disrupt the network.

Overall, PBFT is lightweight under ideal conditions but scales poorly under faults because it keeps broadcasting to all nodes. HotStuff maintains the lowest CPU and network usage in both environments, but its simplified pipeline design increases latency and finalization time. rPBFT consumes more resources in stable networks because it proactively manages fault risk, yet it maintains controlled resource usage and avoids the severe overhead that PBFT incurs under failures. These results indicate that rPBFT achieves a balanced trade-off between proactive fault resilience and scalability, making it more practical than PBFT and more robust than HotStuff for real-world blockchain systems where faults and latency variation are common.

3) *Evaluation Results of Fault Tolerance:* This study evaluated the fault tolerance of rPBFT against PBFT and HotStuff by analyzing throughput under two fault conditions: varying the Byzantine node ratio and the network fault ratio.

In the first experiment, the proportion of Byzantine nodes was gradually increased from 0% to 50%, and the resulting throughput is shown in Fig. 12. rPBFT maintained significantly higher throughput than PBFT and HotStuff in the low-fault range (0–25%), owing to its reliability-based voting adjustment that limits the influence of malicious nodes. It sustained stable operation up to about 30% Byzantine nodes, while PBFT and HotStuff approached their tolerance limit near the conventional 33% threshold. Beyond this point, rPBFT's reliability-based committee became unstable, causing a rapid throughput to

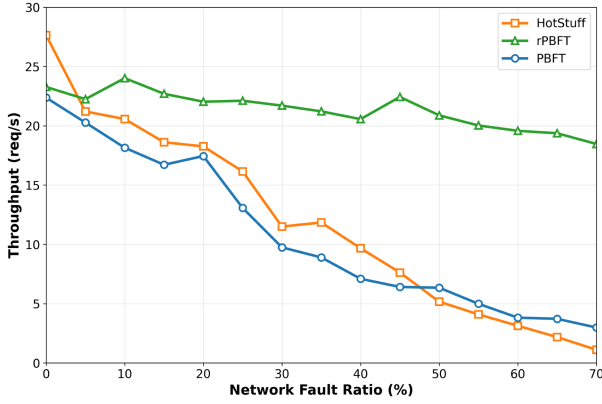


Fig. 13. Throughput variation according to network fault ratio.

collapse to zero. Within practical Byzantine fault ranges, rPBFT remained more efficient and stable than the other algorithms.

The second experiment evaluated the effect of network-level faults such as latency, packet loss, and node unresponsiveness, with fault recovery modeled using the reliability framework; results are shown in Fig. 13. Here, rPBFT consistently outperformed PBFT and HotStuff across all fault ratios, maintaining relatively high throughput even as network faults increased to 60%. This resilience arises from dynamically reducing the participation weight of unreliable nodes and avoiding costly retransmissions. PBFT's throughput fell sharply as faults increased owing to excessive message flooding and repeated view changes, while HotStuff performed better but dropped steeply beyond 50% network faults. Overall, rPBFT combined strong fault tolerance and stable throughput under both Byzantine and network faults, reducing the impact of unreliable or slow nodes and enhancing resilience in heterogeneous, failure-prone environments. Although its consensus collapses beyond about 30% Byzantine nodes, rPBFT achieves the highest efficiency and reliability in practical fault ranges and demonstrates how trust-based dynamic voting can strengthen real-world distributed consensus.

VII. CONCLUSION

Blockchain technology is increasingly applied not only in finance but also in logistics, culture, art, and healthcare as a key infrastructure of the Fourth Industrial Revolution. Practical Byzantine Fault Tolerance (PBFT) offers advantages such as high TPS, absolute finality, and low energy consumption but suffers from limited scalability owing to quadratic message complexity, making it unsuitable for large-scale distributed networks.

This study proposed the reliable PBFT (rPBFT) consensus algorithm, which enhances scalability and reliability by dynamically evaluating node trustworthiness and forming adaptive consensus committees. Through reliability-aware prioritization and committee-based consensus, rPBFT reduces unnecessary broadcasts, mitigates the impact of faulty nodes, and maintains stable performance in heterogeneous network conditions.

Extensive experiments compared rPBFT with PBFT and HotStuff under two fault scenarios: Byzantine node faults and

network-level faults. rPBFT maintained higher throughput than PBFT and HotStuff when the Byzantine node ratio was below 30%, exhibiting slower performance degradation within practical fault ranges. Under network faults such as latency and packet loss, rPBFT consistently outperformed both baselines by dynamically isolating unreliable nodes and adapting its committee structure, sustaining stable throughput even under 60% network communication faults. However, in fault-free environments, rPBFT introduced additional overhead owing to reliability scoring and committee management, resulting in lower efficiency than PBFT and HotStuff. Moreover, consensus stability collapsed when Byzantine nodes exceeded about 30%, as the reliability-based committee could no longer maintain a majority of honest nodes.

This study has some limitations. First, the evaluation focused mainly on probabilistic and non-malicious faults, with limited testing under active adversarial scenarios such as Sybil or coordinated DoS attacks. Second, key parameters such as committee size (c) and the decay factor (α) were chosen empirically rather than through formal optimization, leaving potential for performance tuning in different network conditions. Future work will address these limitations by conducting large-scale experiments on cloud-based testnets, including realistic Sybil and DoS attack simulations. We also intend to design adaptive parameter optimization algorithms that automatically adjust committee size and reliability decay factors according to network stability. Additionally, further analysis of space complexity and convergence time in networks with thousands of nodes will be conducted, along with the development of a security-hardening model to defend against manipulation of committee composition and falsification of reliability scores.

By acknowledging these limitations and outlining a clear roadmap for improvement, this work provides a foundation for building more robust, adaptive, and security-aware consensus protocols for practical blockchain deployment in large-scale and failure-prone distributed environments.

REFERENCES

- [1] P. Dutta, T.-M. Choi, S. Somani, and R. Butala, "Blockchain technology in supply chain operations: Applications, challenges and research opportunities," *Transp. Res. Part E*, vol. 142, 2020, Art. no. 102067.
- [2] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," whitepaper, 2008. [Online]. Available: <https://bitcoin.org/bitcoin.pdf>
- [3] L. Lamport, R. Shostak, and M. Pease, "The Byzantine generals problem," in *Concurrency: The Works of Leslie Lamport*. New York, NY, USA: ACM, 2019, pp. 203–226.
- [4] J.-D. Lovelock, M. Reynolds, B. Granetto, and R. Kandaswamy, "Forecast: Blockchain business value, worldwide, 2017–2030," Gartner, Stamford, CT, USA, 2017. [Online]. Available: <https://www.gartner.com/en/documents/3627117>
- [5] S. Viswanathan and A. Shah, "The scalability trilemma in blockchain," medium, 2018. [Online]. Available: <https://aakash-111.medium.com/the-scalability-trilemma-in-blockchain-75fb57f646df>
- [6] J.-H. Nam, S.-J. Lee, S.-P. Park, and I.-G. Lee, "IP dLedger-decentralized ledger for intellectual property administration," *Technol. Forecasting Social Change*, vol. 186, 2023, Art. no. 122105.
- [7] S.-W. Yun, E.-Y. Lee, and I.-G. Lee, "Selective layered blockchain framework for privacy-preserving data management in low-latency mobile networks," *J. Internet Technol.*, vol. 24, no. 4, pp. 881–891, 2023.
- [8] M. Castro et al., "Practical Byzantine fault tolerance," *OsDI*, vol. 99, no. 1999, pp. 173–186, 1999.

- [9] P. Li, G. Wang, X. Chen, F. Long, and W. Xu, "Gosig: A scalable and high-performance byzantine consensus for consortium blockchains," in *Proc. 11th ACM Symp. Cloud Comput.*, 2020, pp. 223–237.
- [10] Y. Yang, "LinBFT: Linear-communication Byzantine fault tolerance for public blockchains," 2018, arXiv:1807.01829.
- [11] S. Biswas, K. Sharif, F. Li, S. Maharjan, S. P. Mohanty, and Y. Wang, "PoBT: A lightweight consensus algorithm for scalable IoT business blockchain," *IEEE Internet Things J.*, vol. 7, no. 3, pp. 2343–2355, Mar. 2020.
- [12] W. Tong, X. Dong, and J. Zheng, "Trust-PBFT: A PeerTrust-based practical byzantine consensus algorithm," in *Proc. Int. Conf. Netw. Netw. Appl.*, 2019, pp. 344–349.
- [13] G. Xu et al., "SG-PBFT: A secure and highly efficient distributed blockchain PBFT consensus algorithm for intelligent Internet of Vehicles," *J. Parallel Distrib. Comput.*, vol. 164, pp. 1–11, 2022.
- [14] H.-S. Heo and D.-Y. Seo, "A study on scalable PBFT consensus algorithm based on blockchain cluster," *J. Inst. Internet, Broadcast. Commun.*, vol. 20, no. 2, pp. 45–53, 2020.
- [15] M. J. Fischer, N. A. Lynch, and M. S. Paterson, "Impossibility of distributed consensus with one faulty process," *J. ACM*, vol. 32, no. 2, pp. 374–382, 1985.
- [16] D. Mingxiao, M. Xiaofeng, Z. Zhe, W. Xiangwei, and C. Qijun, "A review on consensus algorithm of blockchain," in *Proc. IEEE Int. Conf. Syst., Man, Cybern.*, 2017, pp. 2567–2572.
- [17] Y. Xiao, N. Zhang, W. Lou, and Y. T. Hou, "A survey of distributed consensus protocols for blockchain networks," *IEEE Commun. Surveys Tut.*, vol. 22, no. 2, pp. 1432–1465, Secondquarter 2020.
- [18] S. Zhang and J.-H. Lee, "Analysis of the main consensus protocols of blockchain," *ICT Exp.*, vol. 6, no. 2, pp. 93–97, 2020.
- [19] Z.-C. Li, J.-H. Huang, D.-Q. Gao, Y.-H. Jiang, and L. Fan, "ISCP: An improved blockchain consensus protocol," *Int. J. Netw. Secur.*, vol. 21, no. 3, pp. 359–367, 2019.
- [20] M. M. Jalalzai, C. Busch, and G. G. Richard, "Proteus: A scalable BFT consensus protocol for blockchains," in *Proc. Int. Conf. Blockchain*, 2019, pp. 308–313.
- [21] M. M. Jalalzai, C. Busch, and G. G. Richard, "Consistent BFT performance for blockchains," in *Proc. 49th Annu. IFIP Int. Conf. Dependable Syst. Netw.—Supplemental*, 2019, pp. 17–18.
- [22] M. Yin, D. Malkhi, M. K. Reiter, G. G. Gueta, and I. Abraham, "Hotstuff: BFT consensus with linearity and responsiveness," in *Proc. ACM Symp. Princ. Distrib. Comput.*, 2019, pp. 347–356.
- [23] D. Malkhi and K. Nayak, "HotStuff-2: Optimal two-phase responsive BFT," *Cryptol. ePrint Arch.*, 2023.
- [24] M. A. Rashid and H. H. Pajooh, "A security framework for IoT authentication and authorization based on blockchain technology," in *Proc. 18th IEEE Int. Conf. Trust, Secur. Privacy Comput. Commun./Proc. 13th IEEE Int. Conf. Big Data Sci. Eng.*, 2019, pp. 264–271.
- [25] W. Li, C. Feng, L. Zhang, H. Xu, B. Cao, and M. A. Imran, "A scalable multi-layer pBFT consensus for blockchain," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 5, pp. 1146–1160, May 2021.
- [26] S. Gao, T. Yu, J. Zhu, and W. Cai, "T-PBFT: An eigentrust-based practical Byzantine fault tolerance consensus algorithm," *China Commun.*, vol. 16, no. 12, pp. 111–123, 2019.
- [27] G. Fortino, F. Messina, D. Rosaci, and G. M. Sarné, "Using blockchain in a reputation-based model for grouping agents in the Internet of Things," *IEEE Trans. Eng. Manag.*, vol. 67, no. 4, pp. 1231–1243, Nov. 2020.
- [28] W. She, Q. Liu, Z. Tian, J.-S. Chen, B. Wang, and W. Liu, "Blockchain trust model for malicious node detection in wireless sensor networks," *IEEE Access*, vol. 7, pp. 38947–38956, 2019.
- [29] J. Chen, S. Yao, Q. Yuan, K. He, S. Ji, and R. Du, "CertChain: Public and efficient certificate audit based on blockchain for TLS connections," in *Proc. IEEE Conf. Comput. Commun.*, 2018, pp. 2060–2068.
- [30] T.-H. Kim et al., "A novel trust evaluation process for secure localization using a decentralized blockchain in wireless sensor networks," *IEEE Access*, vol. 7, pp. 184133–184144, 2019.
- [31] S. Li, H. Xiao, H. Wang, T. Wang, J. Qiao, and S. Liu, "Blockchain dividing based on node community clustering in intelligent manufacturing CPS," in *Proc. Int. Conf. Blockchain*, 2019, pp. 124–131.
- [32] U. Khakurel, D. Rawat, and L. Njilla, "Fastchain: Lightweight blockchain with sharding for internet of battlefield-things in NS-3," in *Proc. IEEE Int. Conf. Ind. Internet*, 2019, pp. 241–247.
- [33] B. Choi, J.-y. Sohn, D.-J. Han, and J. Moon, "Scalable network-coded PBFT consensus algorithm," in *Proc. IEEE Int. Symp. Inf. Theory*, 2019, pp. 857–861.
- [34] J. Moubray, *Reliability-Centered Maintenance*. New York, NY, USA: Industrial Press, 2001.



Eunyoung Lee received the BS degree in convergence security engineering and the MS degree in future convergence technology engineering from Sungshin Women's University (SWU), Seoul, Korea, in 2021 and 2023, respectively. She is currently with KB Life Partners, where she works in the field of information security and privacy. Her research interests include the area of convergence security, information security, privacy, and blockchain.



Namryeong Kim received the BS degree in convergence security engineering in 2023 from Sungshin Women's University (SWU), Seoul, Korea, where she is currently working toward the MS degree in convergence security engineering. She is also a researcher with CSE Laboratory, SWU. Her research interests include the areas of convergence security, blockchains, Web3 security, automated vulnerability detection, AI security.



Chaerim Han received the BS degree in convergence security engineering from Sungshin Women's University, Seoul, Korea, in 2024. Her research interests include blockchains and convergence security.



Nayeon Shin received the BS degree in convergence security engineering and the MS degree in future convergence technology engineering from Sungshin Women's University (SWU), Seoul, Korea, in 2021 and 2023, respectively. She is currently conducting information security consulting with a company called Tndsoft. Her research interests include convergence security, information security compliance and Privacy. She has authored or coauthored more than five technical papers in convergence security, network security, and AI security.



Ilgu Lee (Member, IEEE) received the BS degree in electrical engineering from Sogang University, Seoul, Korea, in 2003, and the MS degree from the Department of Information and Communications Engineering from Korea Advanced Institute of Science and Technology (KAIST), Daejeon, Korea, in 2005, and the PhD degree from the Graduate School of Information Security in Computer Science and Engineering Department, KAIST, in 2016. He is currently a professor with the Department of Convergence Security Engineering, Sungshin Women's University, Seoul, Korea. Before joining SWU in March 2017, he was with the Electronics and Telecommunications Research Institute (ETRI) as a senior researcher from 2005 to 2017, and was the principal architect and project leader with Newratek (KR) and Newracom (U.S.) from 2014 to 2017. He has authored or coauthored more than 250 technical papers in the areas of information security, wireless networks, and communications, and holds approximately 200 patents. His research interests include the area of wireless/mobile networks, focusing on information security, computer networks, wireless circuits, and systems. He is also an active contributor to the IEEE 802.11 WLAN standardization committee.